Lab 3: Programming a Real Processing-in-Memory Architecture

Introduction

In this lab, I performed an analysis of a real Processing-in-Memory (PIM) architecture. This involved implementing and evaluating four distinct tasks for the UPMEM PIM System: characterizing data transfer performance, scaling the AXPY vector operation, assessing performance across different data types and operations, and exploring intra-DPU synchronization primitives for parallel vector reduction. Each task was simulated using the simulator provided by the UPMEM SDK.

Task 1 Transferring Data between Main Memory and PIM-enabled Memory

This task was concerned with the data transfer bandwidth between the host CPU and the DPUs. Aside from computation inputs and outputs being transferred between the host CPU and the DPUs, communication between DPUs has also to be routed through the host as an intermediary. Therefore having high transfer bandwidth is important for the PIM system to be effective. Documentation for runtime communication primitives is available in the UPMEM manual.

Implementation Details

There are three different data transfer methods: `SERIAL`, `PARALLEL` and `BROADCAST`. Data transfers have to be invoked on the host, regardless of the transfer direction (CPU to DPU or DPU to CPU). In contrast to the former two methods, `BROADCAST` can only distribute data from the host to the DPUs. The SERIAL and PARALLEL methods rely on the `dpu_prepare_xfer` function. The `BROADCAST` method relies on the `dpu_broadcast_to` function. In my implementation, the data is written to and read from each DPU's `DPU_MRAM_HEAP_POINTER`. The MRAM capacity of each DPU limits the theoretical maximum transfer size per DPU to 64MB. Each transfer size has to be aligned to 8 bytes.

`SERIAL` `SERIAL` passes the `DPU_XFER_DEFAULT` option, which indicates that the transfer should happen synchronously. Although the transfer between the host and the dpus occurs in parallel, each call to `dpu_push_xfer` is blocking. Later calls to `dpu_push_xfer` can only start once the first call has completed. Note there can an alternative way of implementing serial communication, depending on how the task description is understood. This involves iterating over the dpus individually and transferring the data one-by-one. Both interpretations are reasonable, so I implemented the first one based on the fact that it was less effort to program and intuitively performed better.

`PARALLEL` `PARALLEL` passes the `DPU_XFER_ASYNC` option, requesting an asynchronous transfer. This also starts a parallel transfer, but it allows for subsequent transfers to start before every single parallel transfer of the first call has completed. To ensure data integrity in later computation, `dpu_sync` has to be called after a parallel transfer to wait for every asynchronous operation to have completed.

`BROADCAST` `BROADCAST` calls `dpu_broadcast_to`, which copies the same buffer to all DPUS in the set.

```
#ifdef SERIAL   // Serial transfers
    DPU_ASSERT(dpu_push_xfer(
        dpu_set,                            // the identifier of the DPU set
        DPU_XFER_TO_DPU,                    // direction of the transfer
        DPU_MRAM_HEAP_POINTER_NAME,         // location of symbol where to place data
        0,                                  // byte offset from symbol address
        input_size_dpu_8bytes * sizeof(T),  // number of bytes to copy
        DPU_XFER_DEFAULT                    // options of the transfer
    ));

#elif BROADCAST // Broadcast transfers
    DPU_ASSERT(dpu_broadcast_to(
        dpu_set,
        DPU_MRAM_HEAP_POINTER_NAME,
        0,
        bufferX,
```

```
        input_size_dpu_8bytes * sizeof(T),
        DPU_XFER_DEFAULT
    ));

#else // Parallel transfers
    DPU_ASSERT(dpu_push_xfer(
        dpu_set,
        DPU_XFER_TO_DPU,
        DPU_MRAM_HEAP_POINTER_NAME,
        0,
        input_size_dpu_8bytes * sizeof(T),
        DPU_XFER_ASYNC
    ));

    // wait for every asynchronous operation to have been performed
    DPU_ASSERT(dpu_sync(dpu_set));
#endif
```

Evaluation

To evaluate the effectiveness of each transfer method, I transferred data between the host and between 1 and 64 DPUs at three different transfer sizes. The final result per DPU count was an average gained from ten separate runs. The transfer bandwidth is calculated as the total size of the transfer, divided by transfer duration.

Distributing data from the CPU to DPUs As can be seen in figure [@fig:tf_bw], the broadcast method consistently provides the highest bandwidth in the CPU to DPU transfers, while the serial and parallel methods perform similarly overall. The performance benefit between broadcasting and the other two methods is more pronounced for larger transfer sizes. At transfer sizes of 24MB and 48MB, the measured bandwidth increases sharply till around 8 DPUs and then drops off linearly thereafter at a much slower rate.

Collecting data from the DPUs In this scenario, the highest bandwidth is measured at DPU counts less than 5. Thereafter, the bandwidth remains at an almost constant level for the remaining DPU counts. There is very little difference between the serial and parallel transfer methods.

Analysis and Observations

- CPU→DPU is consistently faster than DPU→CPU.
- `SERIAL` and `PARALLEL` look similar here because the asynchrony only helps when you queue multiple outstanding transfers before waiting. In our implementation we issue one transfer and then immediately call `dpu_sync`, which collapses the overlap and makes it behave close to the synchronous version. To expose a benefit, we would need to pipeline several transfers (e.g., multiple chunks or directions) before the sync.
- `BROADCAST` delivers the highest CPU→DPU bandwidth, indicating that fixed overheads are better amortized compared to the other transfer methods. <!– Why the curve peaks early and then declines roughly linearly (more visible at 24–48 MB):

1) Latency amortization then saturation. With a few DPUs, adding more increases parallelism of DMA/transfer engines and quickly fills the shared path, so aggregate bandwidth rises. Once the shared bottleneck (host memory bandwidth, link/interface to the DPU ranks, or driver submission rate) saturates, more DPUs only add contention.
2) Per-DPU fixed overheads. Each additional DPU incurs a relatively constant setup/handshake cost (descriptor preparation, queueing, validation, completion handling). If the payload per DPU is fixed, total time ~ payload_time + N × setup_time, which makes measured bandwidth (total_bytes / total_time) fall approximately linearly in N beyond the saturation point.
3) Broadcast specifics. For broadcast, the host still processes N completions/acks even if the data payload was issued once. As N grows, these control-path costs dominate after the peak, producing the gradual decline.
4) Host effects at scale. More DPUs imply more host-side buffers and metadata, which can increase cache/TLB pressure and reduce copy efficiency, further flattening or lowering throughput after the peak. –>

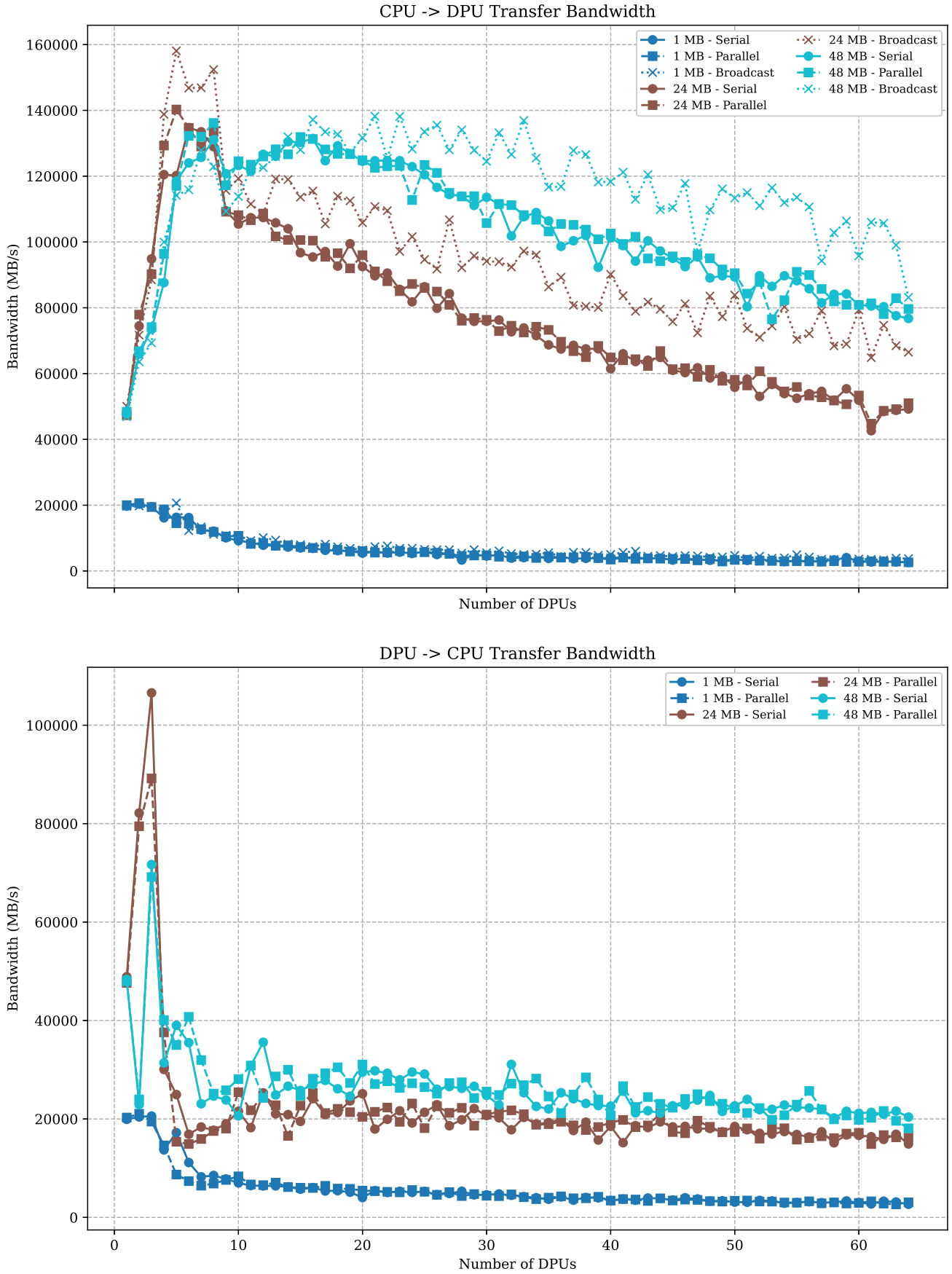

Figure 1: Overall bandwidth vs. number of DPUs for three transfer sizes. Top: CPU→DPU (Serial, Parallel, Broadcast). Bottom: DPU→CPU (Serial, Parallel).

Task 2 AXPY

This task is concerned with quantifying the performance scaling of PIM threads (tasklets). To this end, the AXPY operation ($y = y + alpha\ddot{O}x$) is computed in a distributed fashion on the DPUs.

Implementation Details

After placing both input vectors x and y in the DPUs MRAM heap, the host triggers the distributed computation. Each tasklet allocates a section of memory of fixed size in the DPUs WRAM using `mem_alloc`. Then disjoint blocks of the vectors are copied from the MRAM into the previously allocated tasklet WRAM space using `mram_read`. Special care must be taken to ensure this transfer's size is between 8 and 2048 bytes and aligned by 8 bytes. The AXPY operation is performed on the data block in WRAM and subsequently written back to MRAM using `mram_write`. To conclude, the host copies and aggregates the partial results from each DPU.

Due to hardware limitations, the maximum number of tasklets per DPU is 24. The allocated tasklet WRAM block's size has to be chosen such that each tasklet's data can fit inside the WRAM, which becomes an issue at larger numbers of tasklets. It is beneficial to maximize the block size, since this reduces the copying transfer overheads between the MRAM and WRAM. I found the highest possible block size to be 512B, using at most `2 * 24 * 512B = 24'576B` of the `64KB` of WRAM, which left enough memory for the remaining variables. This configuration produces correct results for any number of tasklets and DPUs.

Evaluation

In [@fig:inst_vs_tlet_cnt], the number of executed instructions per tasklet is plotted against the number of tasklets. For this experiment, two 16MB `uint32_t` input vectors were distributed among 32 DPUS. Scaling the number of tasklets reduces the instructions executed on each tasklet in a non-linear fashion.

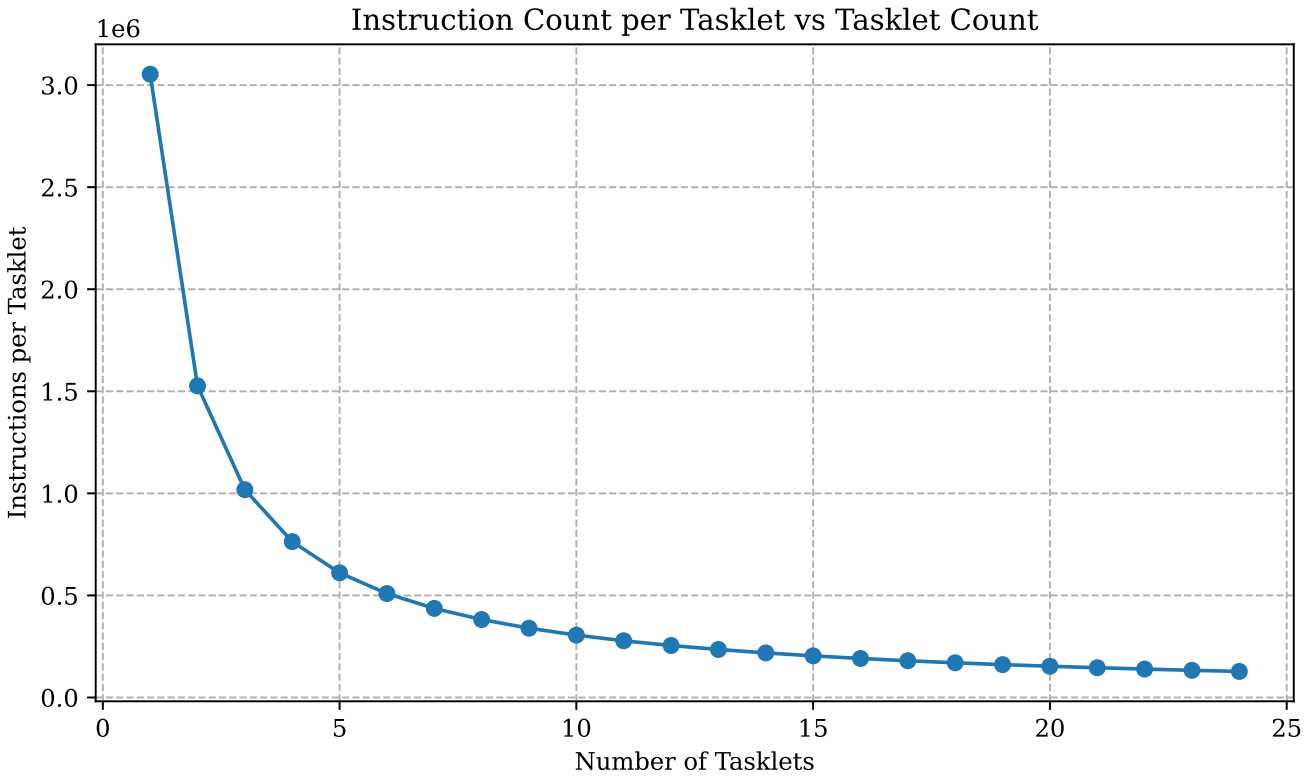

Figure 2: lol

Analysis and Observations

If there were no cost behind copying the data

Task 3 Operations and Data Types

Implementation Details

Evaluation

Analysis and Observations

Task 4 Vector Reduction

Implementation Details

Evaluation

Analysis and Observations

Citations